



PDF Download
3712256.3726461.pdf
18 March 2026
Total Citations: 1
Total Downloads: 1079

Latest updates: <https://dl.acm.org/doi/10.1145/3712256.3726461>

RESEARCH-ARTICLE

Rule-based Machine Learning: Separating Rule and Rule-Set Pareto-Optimization for Interpretable Noise-Agnostic Modeling

GABRIEL LIPSCHUTZ-VILLA, University of Pennsylvania, Philadelphia, PA, United States

HARSH BANDHEY, Cedars-Sinai Medical Center, Los Angeles, CA, United States

RUONAN YIN

MALEK KAMOUN, University of Pennsylvania, Philadelphia, PA, United States

RYAN URBANOWICZ, Cedars-Sinai Medical Center, Los Angeles, CA, United States

Open Access Support provided by:

Cedars-Sinai Medical Center

University of Pennsylvania

Published: 13 July 2025

Citation in BibTeX format

GECCO '25: Genetic and Evolutionary
Computation Conference
July 14 - 18, 2025
Malaga, Spain

Conference Sponsors:
SIGEVO

Rule-based Machine Learning: Separating Rule and Rule-Set Pareto-Optimization for Interpretable Noise-Agnostic Modeling

Gabriel Lipschutz-Villa
gabelip@sas.upenn.edu
University of Pennsylvania
Philadelphia, PA, USA

Harsh Bandhey
harsh.bandhey@cshs.org
Cedars-Sinai Medical Center
Los Angeles, CA, USA

Ruonan Yin
yinruonan@gmail.com
Independent Consultant
Los Angeles, CA, USA

Malek Kamoun
malekkam@pennmedicine.upenn.edu
University of Pennsylvania
Philadelphia, PA, USA

Ryan J. Urbanowicz^{*†}
ryan.urbanowicz@cshs.org
Cedars-Sinai Medical Center
Los Angeles, CA, USA

Abstract

Rule-based machine learning (RBML) algorithms, e.g. learning classifier systems (LCSs), can capture complex relationships while yielding more interpretable models than most other machine learning algorithms. Traditional LCSs rely on a single fitness function for both rule and/or rule-set optimization. However, ideal rule vs. rule-set discovery often requires distinct and multiple objectives. Recently, hybrid-LCSs were proposed that explicitly separated the task of rule vs. rule-set discovery but relied on distinct single-objective or weighted multi-objective fitness functions. This study introduces a newly developed Heuristic Evolutionary Rule Optimization System (HEROS) that combines previous LCS innovations aimed at tackling noisy, larger-scale, classification tasks, while adopting separation of rule vs. rule-set evolution. Uniquely, HEROS employs a custom Pareto-front-based multi-objective fitness function (for rule discovery) and NSGA-II-style multi-objective optimization (for rule-set discovery) to solve both clean and noisy-signal classification problems agnostically. Rule discovery is driven by rule-accuracy and instance coverage objectives, while rule-set discovery is driven by prediction accuracy and rule-set size objectives. Using diverse simulated benchmark datasets, i.e. noisy (GAMETES) and clean (MUX), we demonstrate proof-of-principle that HEROS can directly discover accurate, highly-compact, interpretable, and ideal solutions when compared to the established 'ExSTraCS' RBML algorithm, without objective weightings or adjusting hyperparameters.

CCS Concepts

• **Computing methodologies** → **Rule learning; Supervised learning; Genetic algorithms; Instance-based learning; Batch learning; Cross-validation; Model development and analysis.**

^{*}Shared First-Authorship

[†]Corresponding Author



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

GECCO '25, July 14–18, 2025, Malaga, Spain

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1465-8/2025/07

<https://doi.org/10.1145/3712256.3726461>

Keywords

Rule-based Machine Learning, Learning Classifier Systems, Supervised Learning, Interpretability, Multi-objective Optimization

ACM Reference Format:

Gabriel Lipschutz-Villa, Harsh Bandhey, Ruonan Yin, Malek Kamoun, and Ryan J. Urbanowicz. 2025. Rule-based Machine Learning: Separating Rule and Rule-Set Pareto-Optimization for Interpretable Noise-Agnostic Modeling. In *Genetic and Evolutionary Computation Conference (GECCO '25)*, July 14–18, 2025, Malaga, Spain.. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3712256.3726461>

1 Introduction

Model *interpretability* is a quality often mischaracterized or deprioritized in machine learning (ML) research despite its importance in real-world applications where trust, fairness, transferability, and causality are a priority [19]. An interpretable model is one that allows a human to definitively grasp the reasoning behind the model's decision(s) based on the input data [24]. Separately, an ideal ML algorithm could agnostically approach new problems without relying on assumptions for success (e.g. is the signal clean, noisy, simple, or complex). Some algorithms make innate simplifying assumptions (e.g. linearity), and nearly all manage assumptions as 'hyperparameters'. While automated ML systems show promise in navigating such assumptions with trial and error, it comes with significant computational expense [38]. In this study we aspire to develop a supervised ML classification algorithm that prioritizes model interpretability (including compactness), and avoids assumptions, in solving diverse problems with precision and generalizability.

Rule-Based Machine Learning (RBML) algorithms model problems 'piece-wise' via a set of learned relational rules (typically of the IF:THEN form). RBML sub-families include algorithms such as Learning Classifier Systems (LCS) [30], Association Rule Learning [43], Artificial Immune Systems [5], and greedy rule searches, e.g. RIPPER [4], spanning supervised, unsupervised, and reinforcement learning. RBMLs have drawn attention, in part, for their ability to produce human-interpretable solutions (approaching that of decision trees) in contrast with many other ML algorithms that produce opaque solutions that are at best 'explainable' via estimations [24].

LCSs algorithms, in particular, demonstrate impressive performance in complex reinforcement and supervised-learning classification tasks [30, 34]. They are often categorized as Evolutionary Rule-based ML (ERBML), as they mainly use genetic algorithms

(GA) for rule-discovery/optimization. XCS [40] is the most popular LCS, having simplified and added accuracy-based rule-fitness to the original LCS concept [39]. Two primary styles of LCS have been proposed over the years: *Michigan* (MIC) and *Pittsburgh* (PIT) [30]. MIC-style LCSs (e.g. XCS) use incremental-learning (one instance at a time) to evolve a *population* $\{P\}$ of rules with a GA operating at the level of individual rules and the resulting $\{P_{Final}\}$ is the model (following rule filtering or compaction [21, 26]). Differently, PIT-style LCSs (e.g. GAssist [23]) use batch-learning to evolve a population of variable-length *rule-sets* where the GA operates at the level of an entire rule-set, and a single ‘best’ rule-set is returned as the model. The MIC-style has largely dominated research attention; likely due to its lower compute cost, flexibility in reinforcement or supervised learning, and history of tackling more complex problems. However, PIT-style LCSs have the advantage of yielding more compact (thus more interpretable) rule-sets with less rule-overlap. Both styles typically rely on a single fitness function to drive the GA.

A variety of hybrid algorithms have also been proposed seeking to combine the best of both styles. For example, BioHEL [1, 9] differently learns one rule at a time (i.e. iterative rule-learning) where data instances covered by each added rule are removed from the training set, followed by another round of rule discovery. BioHEL’s GA operates at the level of individual rules (like MIC-style) but uses supervised-batch-learning and yields a compact rule-set solution (like PIT-style). BioHEL similarly uses a single fitness function (for rule-discovery), relying on its unique learning approach to form the final rule-set. While effectively applied to biomedical problems, this strategy may introduce implicit assumptions with the progressive removal of covered instances that could impact performance.

Differently, two hybrid *Fuzzy LCSs* have been proposed for supervised fuzzy learning, each combining MIC and PIT styles into a single batch-learning framework, with separate (single-objective or weighted-multi-objective) fitness functions for rule vs. rule-set optimization [16, 25]. The first algorithm, alternates between a PIT-style rule-set discovery phase, and a MIC-like phase dedicated to fuzzy rule mutation until termination criteria are reached [16]. The second algorithm, runs a complete MIC-style rule optimization followed by a complete PIT-style rule-set optimization, also proposing a cooperative-competitive coevolutionary approach for basic GA parameter self-tuning [25]. These works introduced promising ideas for separating rule and rule-set optimization (each with distinct goals). However, while fuzzy classifiers may arguably yield more human-intuitive models for interpretation, the process of fuzzification limits the precision and utility of such algorithms.

Recently, SupRB [13] was proposed as a hybrid MIC/PIT-style LCS for supervised batch learning in regression. SupRB blends ideas from the fuzzy systems, separating and alternating between rule and rule-set optimization with separate weighted multi-objective fitness functions. Rule-optimization was driven by minimizing rule mean squared error and a generality measure, while rule-set optimization was driven by minimizing in-sample mean squared error and the number of rules in the rule set. SupRB was also extended to employ basic parameter self-tuning [12], but as in [25], this did not include objective weight parameters. While promising, SupRB has only yet been evaluated on a small set of UCI ML Repository [8] datasets where the solution ground truth is unknown. Also, the flexibility of this approach to tackle different problems without making

assumptions is likely limited by manual objective weighting. As an alternative, Pareto-optimization has the advantage of not assuming a specific objective weighting [11], which is valuable for solving disparate problems. Previously, a Pareto-inspired multi-objective rule fitness was pioneered for supervised MIC-style LCSs [36].

This study proposes a novel RBML/LCS framework that adopts separation of *rule* vs. *rule-set* discovery/optimization; each driven by distinct adaptive, Pareto-front-based multi-objective optimization strategies. We introduce this Heuristic Evolutionary Rule Optimization System (HEROS), initially targeting supervised binary classification tasks as proof-of-principle, with the goal of compactly, interpretably, and accurately modeling a variety of problems (i.e. clean, noisy, simple, complex) without weighting assumptions or adjusting hyperparameters. HEROS was implemented from the ground up, simplifying aspects of traditional LCSs, and adapting a variety of promising LCS algorithmic elements. It adopts a two-phase LCS paradigm [13, 16, 25], applying aspects of MIC and PIT-styles in turn (i.e. no phase alternation).

Phase I of HEROS focuses on learning a diversity of candidate rules and is directly descended from MIC-style LCSs including XCS [40], UCS (focused on supervised learning) [3], and ExSTraCS (focused on supervised learning in noisy, larger-scale problems) [29, 35]. Phase I introduces new strategies for subsumption-based rule compaction and the prediction array while incorporating (1) batch rule-evaluation [42] (2) expert knowledge guided covering and mutation [31], (3) a mixed discrete-continuous attribute-list rule representation similar to ALKR [2, 29], and (4) an automatic rule-specificity limit [35]. Phase I adapts an improved Pareto-inspired rule fitness function (based on [36]) that optimizes *rules* for trade-offs in maximized *useful rule-accuracy* and/or *correct instance coverage*. This is meant to value a diversity of candidate rules given a spectrum of possible problem ground-truths; i.e. from no/low noise (needing high accuracy rules) to high/moderate noise (needing high correct coverage rules). Since each rule only represents a ‘piece’ of the overall model/solution, this strategy should also avoid undue pressure for all rules to be ‘close’ to the Pareto-front, instead serving as a fitness standard encouraging the discovery and maintenance of diverse rules that may be suited to covering distinct niches of the problem. *Phase II* of HEROS leverages NSGA-II-inspired multi-objective Pareto-optimization [6] to combine Phase I rules (as building blocks) into *rule-sets* (a.k.a. ‘models’) that maximize *balanced accuracy* and/or minimize *rule-set size*. This is meant to identify Pareto-optimal candidate models ranging from highly accurate (but potentially over-fit), to less accurate, but more compact and potentially less-over-fit. Phase II also adds a *merge* operator for the GA, and *random* vs. *target-accuracy* model initialization.

2 Methods

We detail both phases of the HEROS algorithm and describe our experimental evaluation comparing HEROS to ExSTraCS across simulated clean (MUX) and noisy (GAMETES) benchmark datasets.

2.1 HEROS: Overview

HEROS was implemented for structured, supervised data; allowing categorical and/or quantitative features, and binary or multi-class outcomes. It allows for missing data values; treated agnostically as

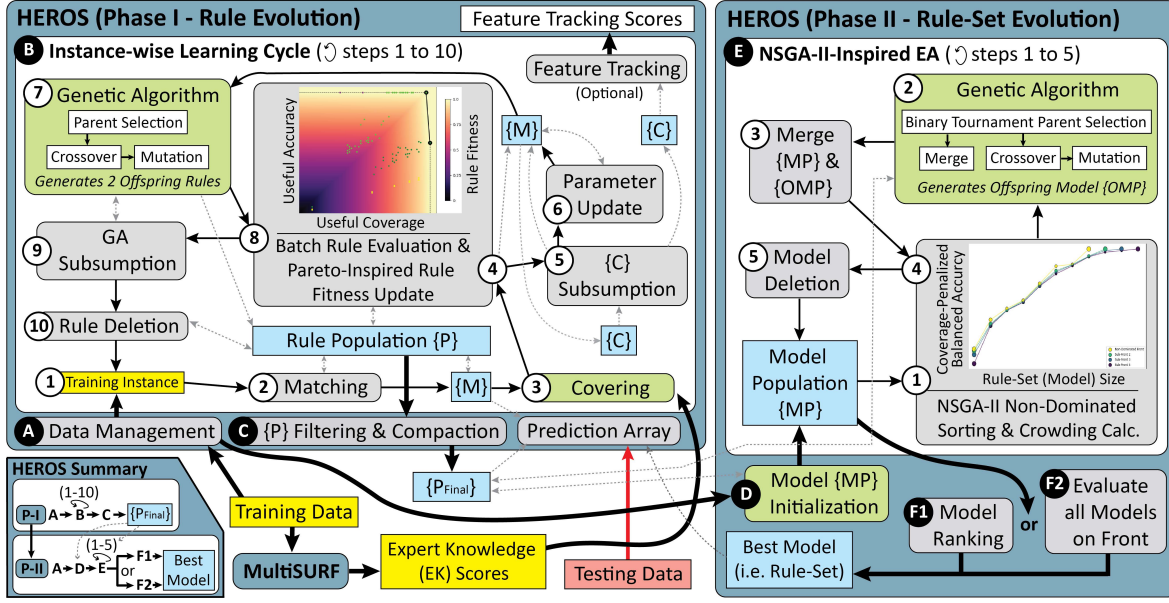


Figure 1: Schematic of HEROS Algorithm (Phases I and II). Bottom-left box summarizes algorithm phases, components (letters), and learning cycle steps (numbers). Yellow boxes indicate training data or EK weights, light-blue boxes indicate rule or model sets, green boxes indicate rule or model discovery elements, and gray boxes indicate other algorithm elements. Thick-black lines indicate algorithm flow between phases and components, thin black lines indicate flow between cycle steps and dotted grey lines indicate element interactions. Phase I outputs a rule population $\{P_{Final}\}$ (akin to a MIC-style LCS model). Phase II outputs a ‘best’ model (drawn from $\{MP\}$), consisting of a sub-set of rules from $\{P_{Final}\}$.

‘not-matched’ by rule-matching for both training and prediction [35]. However, this initial study focuses on data with categorical features, binary outcomes, and no missing values. Description of HEROS’s phases, components, and learning cycles steps follows the algorithm schematic in Figure 1. Both *data management* and *prediction array* components are shared by both phases. Relevant HEROS hyperparameters (and the default values applied herein) are given in Table 1. Of note, the first four hyperparameters can be important to adjust in practice as they limit the computational run time and memory allotted for search and discovery.

2.2 HEROS: Phase I - Rule Optimization

HEROS Phase I is a new MIC-style LCS, most closely inspired by the scikit-ExSTraCS implementation [44, 45]. Following Figure 1, Phase I begins with (A) *data management* which takes training data, identifies feature types, stores data statistics (e.g. p_c ; proportion of instances with a given class, ‘c’), automatically calculates a rule specificity limit as described in [35] (serving as a practical data-driven limit to how many features can be specified within a rule), and scales any expert knowledge (EK) feature weights to the range (0,1) [31]. Instances are shuffled prior to the start of learning.

2.2.1 Rules. Rules are represented as IF:THEN (i.e. condition: action) expressions using a mixed discrete-continuous attribute-list rule representation similar to ALKR [2, 29]. Rules specify features and their corresponding values as the rule’s *condition*. Features not specified by the rule are ignored during *matching* allowing for generalization of the target problem. The rule’s action is simply the

class it predicts. Each rule maintains internal parameters including: (1) N_r , *numerosity*, i.e. number of virtual copies of a rule, (2) M_r , number of training instances *matched* by the rule, (3) C_r , number of training instances the rule both matches and has the same (i.e. correct) action, (4) A_r , *rule accuracy*; calculated as C_r/M_r , (5) UA_r , ‘*useful*’ rule accuracy, i.e. the accuracy of the rule beyond random expectations of p_c scaled within the range (0,1) (see Eq. 1 and [28]), (6) UC_r , ‘*useful*’ rule coverage, i.e. the coverage of the rule beyond random expectations of p_c (see Eq. 2 and [36]), (7) F_r , *rule fitness* (see section 2.2.3), and (8) E_r , estimated average size of match sets where the rule matched (see section 2.2.2).

$$UA_r = \begin{cases} \frac{A_r - p_c}{1 - p_c} & \text{if } \frac{A_r - p_c}{1 - p_c} \geq 0, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

$$UC_r = \begin{cases} C_r - (M_r \times p_c) & \text{if } C_r - (M_r \times p_c) \geq 0, \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

2.2.2 Instance-wise Learning Cycle. Next in Phase I, (B) an instance-wise learning cycle is applied for (Rule_I) iterations. Each iteration, the cycle starts (B-1) by focusing on a single training instance. The rule population $\{P\}$ is initially empty. Next, (B-2) matching is applied to form a set of all rules in $\{P\}$ that match the specified feature value states of the current instance, i.e. $\{M\}$ as previously detailed [35]. If $\{M\}$ is empty or no rules in $\{M\}$ have the correct action, (B-3) a *covering* operator is applied to generate a new rule that matches the current instance. This can be done randomly or via expert knowledge covering, based on whether expert knowledge

Table 1: HEROS Algorithm Hyperparameters.

Hyperparameter	Default	Description
Rule_I	200,000	Number of Phase I learning iterations
Rule_P	2,000	Maximum rule population size
Model_I	200	Number of Phase II learning iterations
Model_P	100	Maximum model population size
ν	1	Accuracy pressure (1 is neutral)
β	0.2	Average $\{M\}$ size learning rate
θ_{sel}	0.5	$\{M\}$ proportion for tournament sel.
p_cross	0.8	Probability of applying crossover
p_mut	0.04	Probability of applying mutation
p_merge	0.1	Probability of applying merge
new_gen	1.0	Controls size of $\{OMP\}$ as $f(\text{Model_P})$
model_pop_init	random	Model initialization method
subsumption	both	Use GA, $\{C\}$, or 'both' subsumptions
rs1	auto	Manually specify rule specificity limit
compaction	sub	Rule compaction method
random_state	(0,20)	Applied random seed

(EK) feature scores were provided (see [31]). In this study, we generated EK scores using MultiSURF [37], an interaction-sensitive feature importance scoring algorithm. New rules are initialized with $N_r = 1$. Unlike ExStracs [35], during the subsequent rule evaluation step, HEROS calculates rule UA_r for all possible actions, and selects the action with the highest UA_r . This difference from ExSTraCS decreases algorithm effort exploring poor quality rules, but also leads to the possibility of $\{C\}$ being empty during some Phase I iterations. The 'current instance' class is assigned as the action in the case of tied UA_r to promote correct coverage when possible.

Next, also unlike ExSTraCS [35], **(B-4) batch evaluation** is applied to the new rule, using the entire training dataset to update M_r , C_r , A_r , UA_r , and UC_r (simultaneously selecting an action as previously described). Batch evaluation is important for subsequent Pareto-inspired rule fitness calculation which relies on complete evaluation of a given rule. Batch rule evaluation only needs to be done once at the birth of a new rule. This is followed by checking whether the new rule is *non-dominated* on the current rule *Pareto-front*. If it is, the fitness of all rules in $\{P\}$ are updated. Otherwise, only the fitness of the new rule is updated. Details are given in section 2.2.3.

Next, **(B-5) $\{C\}$ -subsumption** is applied. LCS subsumption mechanisms apply an explicit generalization pressure to rule evolution [30]. $\{C\}$ subsumption finds the most accurate and general rule in $\{C\}$ and checks if it can *subsume* any other rules in $\{C\}$. *Subsumed* rules are removed from $\{P\}$, $\{M\}$, and $\{C\}$ and for each, the *numerosity* (N_r) of the *subsumer* is increased by 1. HEROS adopts a stricter implementation of subsumption than ExSTraCS or other previous LCSs, made possible by batch rule evaluation. Specifically, Rule 'A' can subsume Rule 'B' if the following conditions are met:

- (1) Rule 'A' has the same action as Rule 'B'
- (2) $A_{r,A} \geq A_{r,B}$
- (3) Rule 'A' has fewer specified features than Rule 'B'
- (4) Rule 'B' specifies all features specified in Rule 'A'

Next, **(B-6)** HEROS updates E_r for all rules in $\{M\}$ using the Widrow-Hoff algorithm (see Eq. 3), where M_{sum} is the sum of rule N_r in $\{M\}$. This parameter will later influence *deletion* to favor deleting rules that match instances already well covered by other rules

in $\{P\}$. At this point feature tracking scores can (optionally) be updated based on rules in $\{C\}$ to enhance downstream rule-population interpretation [33]. ExSTraCS [35] leverages these scores to guide ongoing rule discovery [27] but HEROS does not.

$$E_r = E_r + \beta(M_{\text{sum}} - E_r) \quad (3)$$

Next **(B-7)** a GA is applied as the primary rule discovery mechanism in a manner similar to ExSTraCS [35], however importantly, HEROS selects parents from $\{M\}$ rather than $\{C\}$. Tournament selection is applied to select two parent rules from $\{M\}$ based on θ_{sel} which are copied to initialize two offspring rules. Uniform crossover is applied between them with probability p_cross followed by niche mutation within each (which only yields rules that match the current training instance). The number of mutations applied to a given rule is controlled by repeatedly generating a random number between (0,1), checking if it is less than p_mut, and if so, a mutation counter (c) (initialized at 0) is increased by 1, and if not this process stops. For each c , HEROS chooses a new feature to specify or generalize in a manner that introduces c total unique changes to the rule. If $c = 0$, HEROS will make a single 'swap' mutation (with probability $1 - p_{\text{mut}}$) that maintains current rule specificity. Overall, when using a lower p_mut (e.g. 0.04), this ExSTraCS-inspired mutation strategy tends to make consistent but smaller changes to offspring rules while occasionally yielding larger changes. Also like ExSTraCS, HEROS uses EK to probabilistically guide mutation such that features with higher EK scores are more likely to be specified in rules (and vice versa).

Next, **(B-8)** each offspring rule is (1) batch evaluated, (2) assigned the action yielding the highest UA_r (as with covering), (3) checked for Pareto non-domination, (4) and has fitness calculated as detailed in section 2.2.3. This is followed by **(B-9) GA subsumption** that is almost identical to $\{C\}$ subsumption, except it checks whether either parent rule subsumes either offspring rule. Any remaining offspring rules are added to $\{P\}$ if it don't already exist in $\{P\}$. If it does, the *numerosity* of the existing rule is increased.

Lastly, **(B-10) rule deletion** is applied to limit the N_r sum of rules in $\{P\}$ (i.e. micro-population size) to the maximum population size (Rule_P). Like ExSTraCS, HEROS Phase I uses roulette wheel selection to select rules for deletion [35]. This involves the calculation of deletion weights for all rules (D_r) [17]. Different from ExSTraCS and most other MIC-style LCSs, Phase I simplifies the deletion weight calculation (see Eq. 4) since HEROS rules are batch evaluated and have equal experience. A rule's probability of deletion is inversely proportional to it's fitness, however rules with high N_r or E_r have an increased likelihood of deletion. After completing a learning cycle, Phase I gets the next available instance, or returns to the first, if all training instances have been examined.

$$D_r = \begin{cases} (E_r \times N_r^2)/0.001 & \text{if } F_r = 0, \\ (E_r \times N_r^2)/F_r & \text{otherwise.} \end{cases} \quad (4)$$

2.2.3 Pareto-Inspired Rule Fitness. One key to HEROS functionality is its Pareto-inspired multi-objective rule fitness, first pioneered in [36], but improved herein. Pareto-optimization [11] is critical to our goal of avoiding assumptions in real-world applications, given that for any new problem/dataset, there is often unknown expectations of signal vs. noise and/or the nature of underlying association(s),

e.g. the number of relevant features or pattern(s) of association such as linear, interactive, additive, or heterogeneous. Given that each rule only represents a potential solution ‘piece’; maintaining a Pareto-front of discovered non-dominated rules serves as an *adapting* standard for evaluating F_r without assigning objective weight assumptions. HEROS Phase I seeks to optimize rules for two objectives: (1) maximize ‘useful’ rule accuracy (UA_r) and (2) maximize ‘useful’ rule coverage (UC_r). The UC_r objective is selected over potential alternatives such as rule specificity, M_r , or C_r , since we want each objective to have a clear gradient from ‘undesirable’ to ‘desirable’ across target problems. Specifically, (1) minimizing the number of features specified in a rule is problematic, given that the ideal value would be inconsistent across problems, (2) maximizing M_r can lead to sub-optimally prioritizing over-general rules, and (3) maximizing C_r can place disproportionate value on rules with the majority-class *action* given an imbalanced data. UC_r blends the concepts of coverage (i.e. rule generality) and accuracy to define a more reliable gradient while also accounting for class imbalance.

Following batch rule evaluation, we check whether the UA_r and UC_r objective pair dominates any rule objective pair on the current non-dominated front. Note that HEROS is initialized with an empty front. Rule ‘A’ dominates Rule ‘B’ if A is at least as good as B in both objectives and strictly better in at least one objective. If the new rule is non-dominated, i.e. Pareto-optimal (PO), it’s added to the front, and any (now) dominated rules on the front are removed (but not deleted from $\{P\}$). Updates to this front will trigger a recalculation of all F_r in $\{P\}$ since the fitness function has adapted. Figure 1 (B–8) and Appendix A.1.2 visualize a rule PO front and the underlying fitness gradient it determines. The two larger black dots, joined by a solid line identify two non-dominated rules (discovered in that example). The dotted lines that horizontally span to the maximum UA_r (i.e. UA_{max}) or vertically span to the maximum UC_r (i.e. UC_{max}) are not on the PO front, but serve as additional segments of an overall ‘reference front’ for fitness calculation.

Improving upon [36], HEROS calculates F_r as a function of distance to the front (d_r), i.e. the distance from a rule’s objective pair (point) to the closest point on any segment of the ‘reference front’, with a maximum distance of 1. While UA_r has a maximum of 1, UC_r objectives are scaled to (0,1) by UC_r/UC_{max} for distance calculations. Eqs. 5 and 6 define fitness adjustments yielding a small added fitness gradient for rules on (or closest to) non-PO segments of the reference front (with penalty term $p = 0.01$). Eq. 7 defines F_r for a given rule (r) including special cases such as $v > 1$ and/or when the closest line segment is not PO (e.g. ‘if UA_{max} nearest’). This creates a smooth fitness gradient with highest fitness for rules on (or close to) the PO front, but still high fitness for any rule close to non-PO segments of the reference front.

$$C_a = (1 - p) + p\left(\frac{UC_r}{UC_{max}}\right) \quad (5)$$

$$A_a = (1 - p) + p\left(\frac{UA_r}{UA_{max}}\right) \quad (6)$$

This *adaptive fitness function* is aimed at finding a diversity of rules with the best trade-offs among objectives to be used as Phase II building blocks. Therefore, we expect Phase I to (A) yield some high fitness rules that ultimately will be non-ideal for rule-set inclusion, (B) yield some lower/moderate fitness rules that ultimately may be

ideal for rule-set inclusion, and (C) potentially take more learning iterations to find ideal rules in clean problems (even when $v > 1$) in contrast with supervised MIC-style LCSs that rely on a accuracy-based fitness alone. However, this trade-off is expected to give HEROS the problem-adaptability necessary to reliably tackle a diversity of real-world classification tasks.

$$F_r = \begin{cases} 0 & \text{if } UA_r = 0 \text{ \& } UC_r = 0 \\ 1 & \text{if } r \text{ is PO \& } v = 1 \\ C_a & \text{if } UA_r = UA_{max} \\ A_a & \text{if } UC_r = UC_{max} \text{ \& } v = 1 \\ (1 - d_r) * C_a & \text{if } UA_{max} \text{ nearest \& } v = 1 \\ (1 - d_r) * A_a & \text{if } UC_{max} \text{ nearest \& } v = 1 \\ (1 - d_r) & \text{if PO nearest \& } v = 1 \\ (UA_r)^v * A_a & \text{if } r \text{ is PO \& } v > 1 \\ (UC_r)^v * A_a & \text{if } UC_r = UC_{max} \text{ \& } v > 1 \\ (UA_r)^v * (1 - d_r) * C_a & \text{if } UA_{max} \text{ nearest \& } v > 1 \\ (UC_r)^v * (1 - d_r) * A_a & \text{if } UC_{max} \text{ nearest \& } v > 1 \\ (UA_r)^v * (1 - d_r) * A_a & \text{if PO nearest \& } v > 1 \end{cases} \quad (7)$$

2.2.4 Filtering and Compaction. A variety of rule compaction strategies have been proposed for MIC-style LCS, seeking to remove low-performing, redundant, or less experience rules; ideally without reducing classification performance [7, 10, 20, 21, 26, 41]. However, (1) these were all designed to maximally compact $\{P\}$ as a model for making predictions, (2) some assume clean problems e.g. CRA [41], (3) many assume $\{P\}$ is a *complete action mapping* (used in reinforcement learning), and (4) most assume a single ‘accuracy’ objective for F_r . Rule compaction methods designed to accommodate noisy signals would often lead to diminished interpretability of underlying feature relationships despite maintained prediction accuracies [26]. ExSTraCS [35] adopts a quick rule filter (QRF) that only removed obviously poor/inexperienced rules.

The last component of Phase I, (C) involves $\{P\}$ post-processing via rule filtering and compaction to yield a $\{P_{Final}\}$ that includes a diversity of high-quality candidate rules, rather than returning a maximally compact $\{P_{Final}\}$. HEROS first filters out ‘clearly bad rules’, i.e. $UA_r = 0$. Then, inspired by the subsumption aspect of the RCR compaction approaches [20, 21], HEROS compaction efficiently applies $\{P\}$ -wide subsumption to remove all over-specific rules (considered suboptimal regardless of problem-noise assumptions).

2.2.5 Prediction Array. After training, HEROS’s makes class predictions on evaluation data using a prediction array. Different from ExSTraCS and most other LCSs, HEROS abandons the traditional fitness-based rule voting scheme, instead adopting a strategy similar to random forest [14] This is detailed in Appendix A.1.3. The HEROS prediction array is shared by both phases, such that either $\{P_{Final}\}$ from Phase I, or a ‘best’ rule-set from Phase II (discussed later) can be used to make predictions. Evaluation data is never seen by HEROS until both phases complete training. In brief, prediction begins by forming $\{M\}$ for a given instance (from either $\{P_{Final}\}$ or the best rule-set) followed by calculating average posterior class probabilities for each possible class across all matching rules.

2.3 HEROS: Phase II - Model Optimization

Phase II aims to combine candidate rules from $\{P_{Final}\}$ into maximally accurate and compact rule-sets (i.e. models), using an NSGA-II-inspired [6] multi-objective evolutionary algorithm. While Phase I relies on a custom fitness function to account for the uncertainty of rule optimality in the context of a final model, Phase II disposes of solution ‘fitness’ in favor of NSGA-II style fast non-dominated sorting, crowding distance calculations, and front-rank/crowding-distance-based binary tournament parent selection [6]. Phase II inherits the following Phase I parameters: ν , p_{cross} , and p_{mut} .

2.3.1 Models. Each model includes a subset of unique rules from $\{P_{Final}\}$ and internal parameters: (1) A_m , the model ‘balanced’ accuracy; calculated as mean of *sensitivity* and *specificity* across training instances (where any non-covered instances are treated as incorrect predictions to promote model coverage), (2) S_m , the number of rules in the model, (3) C_m , the proportion of instances covered (i.e. matched) by the model, and (4) whether the model is on the current best-ranked model Pareto-front. During Phase II, new models have their A_m evaluated automatically after confirming they do not already exist in the current model population $\{MP\}$ or newly generated offspring model population $\{OMP\}$. Notably, HEROS rule-sets are unordered, and do not use a *default rule* [1, 23]. While a default rule yields more compact rule-sets, we favor a *best action* mapping [18] of the solution for completeness of interpretation.

Importantly, during model training, model prediction evaluations are not made as previously described for the *prediction array*. Instead, each matching rule in the rule-set gets a prediction probability of 1 for the action it asserts (i.e. unweighted by N_r or the underlying posterior class probabilities for that rule). Also, similar to non-covered instances, instances yielding prediction ties here are also counted as incorrect predictions during model optimization. These differences serve as added pressures for models to minimize the number of rules needed to match (i.e. cover) and make accurate predictions on individual instances across the training data. Ultimately, we consider a rule-set to be optimally interpretable when only one rule at a time would match a given training or testing instance, with the exception of problems that may require some degree of rule overlap for optimal performance.

2.3.2 Model Initialization. Phase II begins (Figure 1-D) by initializing $\{MP\}$ of size $(Model_P)$. This study compares two model initialization strategies (*model_pop_init*); *random* (RI) and *target accuracy* (TI). RI generates random unique rule-sets of a size in the range $(10, len(\{P_{Final}\})/2]$. Alternatively, TI expands the RI method by generating each rule-set using a different target A_r between $(0.55 \text{ and } 1.0)$. Given a target A_r this method first tries to fill the model with rules that equal this accuracy, and then probabilistically adds rules that are closest to this accuracy. This way, $\{MP\}$ is initialized with rule sets that span assumptions of noisy to clean problems, i.e. some rule sets may only include rules with $A_r = 1$, and others may only include rules with $A_r \approx 0.75$, etc. When $\nu > 1$, TI initialization always uses a target A_r of 1.0.

2.3.3 NSGA-II-Inspired EA. Following initialization, (E) NSGA-II-inspired optimization cycles are applied for $(Model_I)$ iterations. Each iteration, the cycle starts (E-1) by applying NSGA-II-style fast

non-dominated sorting of all models in $\{MP\}$ into ranked Pareto-dominance fronts based on maximizing A_m and minimizing S_m . This is followed by calculation of crowding distances within each front used to help encourage diversity during parent selection and deletion. Both steps were implemented as described in [6] and visualized in Figure 1 (E-1) and Appendix A.1.2.

Next, (E-2) the GA generates offspring models while the size of $\{OMP\}$ is less than $(Model_P * new_gen)$. First, the GA applies NSGA-II-style binary tournament selection to choose two parent models from $\{MP\}$ as described in [6]. Second, up to 3 offspring models are generated for each pair of parents: one via a *merge* operator (with probability p_{merge}) that combines all unique rules from both parents into one offspring, and others with crossover and mutation operators. While the crossover operator is similar to Phase I, the Phase II mutation operator has a probability of p_{mut} for each rule in the offspring model to either add, remove, or swap a rule that has not yet been changed in that offspring. Furthermore, when $\nu > 1$, higher accuracy rules have a higher probability of being added or swapped into the model, while lower accuracy rules have a higher probability of being removed or swapped out. Additionally, whenever a new offspring model already exists, the GA makes a finite number of attempts to initialize a novel random model using either RI or TI rather than skip adding the duplicate model.

Next, (E-3) $\{OMP\}$ is merged with $\{MP\}$ followed by (E-4) another round of non-dominated model sorting and crowding distance calculation and (E-5) NSGA-II-style deletion [6], where the next generation of size $Model_P$ is selected from all current models by filling $\{MP\}$ with models from the best to worst ranked front. Once a front is reached where all models on that front can’t be added to $\{MP\}$, we add the models from that front with the highest crowding distance until $Model_P$ is reached.

2.3.4 Best Model Selection. Once $\{MP\}$ training has ended, HEROS Phase II offers two strategies, (F1 or F2), to pick a ‘best’ model from the non-dominated front, denoted in the results as *default* (D) or *optimal model front* (F), respectively. Method ‘D’ selects the model with the largest training A_m , subsequently using larger instance coverage and smaller S_m as tie-breakers. Method D can be more susceptible to over-fitting because of its focus on the accuracy objective. Method ‘F’, evaluates the *testing data* performance of all non-dominated models, each treated as a candidate solution. This is in-line with the intended utility of Pareto-optimization, yielding a spectrum of non-dominated candidate solutions for users to select from. Stepping through all non-dominated models, we replace the current best if the next model either (1) has $>$ balanced testing accuracy and $\geq$ testing coverage, or (2) has $\geq$ balanced testing accuracy, $\geq$ testing coverage, and a smaller rule count.

2.4 Experimental Evaluation

As proof-of-principle evaluation of HEROS (Phase I and II) we compare it to ExSTraCS 2.0 [35, 45], across a set of 6 GAMETES-simulated genomic datasets with varying degrees of signal noise (where features can have values 0, 1, or 2) [32] and 5 clean binary-valued n-bit multiplexer (MUX) benchmark datasets [15, 35, 40]. Table 2, identifies their total feature count, # of predictive features, # of instances (pre-CV-partitioning), estimated maximum possible prediction accuracy (based on simulation parameters), type/complexity

Table 2: Simulated GAMETES (A-F) and MUX (6-70) datasets.

Data Name	Tot. Feat.	Pred. Feat.	Inst.	Max. Acc.	Nature of Underlying Association	Optimal Rule Count
A	100	4	1600	≈ 0.94	additive univariate	Unknown
B	100	1	1600	≈ 0.84	univariate	≈ 3
C	100	2	1600	≈ 0.82	2-way epistasis	≈ 9
D	100	4	1600	≈ 0.70	2-het. x 2-way epistasis	≈ 18
E	100	4	1600	≈ 0.65	4-het. x univariate	≈ 12
F	100	3	1600	≈ 0.65	3-way epistasis	≈ 27
MUX6	6	6	500	1	4-het. x 3-way epistasis	8
MUX11	11	11	5000	1	8-het. x 4-way epistasis	16
MUX20	20	20	10000	1	16-het. x 5-way epistasis	32
MUX37	37	37	10000	1	32-het. x 6-way epistasis	64
MUX70	70	70	20000	1	64-het. x 7-way epistasis	128

of association(s), and the optimally compact rule-set size. ‘Het.’ refers to heterogeneous association, and epistasis refers to feature interaction [32, 36]. All datasets differ in terms of underlying problem complexity. Note that for MUX datasets, a fixed number of possible instances were generated for each, which is limiting for solving such problems, but aligns with previous work treating MUX problems as offline supervised learning tasks [35]. Given this, and our relatively low study-fixed settings for (Rule_I), we don’t expect either algorithm to perform well on higher-order MUX problems.

Across this study we consider (1) $\nu = 1$ (no added accuracy pressure) vs. $\nu = 10$ (strong explicit accuracy pressure), for both HEROS and ExSTraCS on MUX datasets and (2) either *random* or *target* model initialization for HEROS Phase II across all datasets. Otherwise, HEROS used the hyperparameter settings in Table 1, and ExSTraCS used the same or closely matched settings, when relevant. See Appendix A.1.1 for ExSTraCS hyperparameter settings, differences, and value selection. Each dataset in Table 2 was pre-partitioned into 10 train/test pairs to ensure analyses were run on identical partitions. Each algorithm configuration was run with 20 different random seeds, yielding 200 algorithm runs for each dataset/algorithm-config. pair (i.e. 9800 runs study-wide).

For each run, we tracked algorithm performance metrics (i.e. balanced testing accuracy, testing data coverage, rule count, and run time) for key configurations and checkpoints: i.e. evaluating (1) **ExSTraCS**; after training + QRF filtering, (2) **H(I)-Pre**; HEROS after Phase I training, (3) **H(I)-Post**; HEROS after Phase I filtering/compaction, and (4) **H(II)**; HEROS after Phase II training. This last checkpoint is further differentiated as **H(II)-RI** or **H(II)-TI** to indicate results when using *random* vs. *target* model initialization, and **-D** or **-F**, to indicate whether the default vs. optimal-model-front method was used to select the ‘best’ model. Statistically significant metric comparisons were identified between ExSTraCS and HEROS using the Wilcoxon Rank sum test ($p < 0.05$).

We compare to ExSTraCS since it is a supervised-learning MIC-style LCS designed to tackle noisier problems, that also previously solved the (6 to 135-bit) set of MUX problems (using $\nu = 10$) with high accuracy, but without finding ideally compact rule-sets [35]. Of note, LCS algorithms; XCSCFC [15] and ConCS [22] have demonstrated highly scalable methods to solve MUX and other complex benchmark Boolean problems. However, these methods rely on (1) a *code fragment* representation rather than traditional rules (which arguably reduces interpretability), and (2) *transfer learning* which adds the assumption that a related and/or simpler version of the

problem has been previously learned. While powerful, these approaches directly contrast with our prioritization of interpretability and assumption-minimizing learning for noisy problems.

The HEROS scikit-learn-compatible base code and study evaluation scripts are available on GitHub (<https://github.com/UrbsLab/heros>). All analyses were conducted on a Linux parallel computing cluster including 384 virtual cores, each with 6GB RAM.

3 Results and Discussion

Table 3 summarizes comparisons between ExSTraCS and HEROS configurations (across Phases I and II) for the GAMETES datasets (A-F). For each metric, the mean (standard deviation) over 10 CVs $\times$ 20 random seeds (i.e. 200 runs) is reported. Up or down arrows indicate significant metric differences ($p < 0.05$) in contrast with ExSTraCS. Rows in yellow highlight the HEROS Phase II configuration yielding the best overall performance. Key observations across noisy GAMETES data evaluations include: (1) With Phase II, HEROS achieves statistically better or comparable testing accuracies to ExSTraCS in all 6 datasets while also massively reducing rule count for each (e.g. 13 vs. 1103 rules for dataset C while simultaneously yielding higher average testing accuracy), (2) HEROS Phase II rule count reduction occasionally yielded minor reductions in testing data coverage, (3)

Table 3: HEROS vs. ExSTraCS on GAMETES Datasets

Data	Algorithm Config.	Testing Accuracy	Testing Coverage	Rule Count	Run Time (Minutes)
A	ExSTraCS	0.929(0.018)	1.0(0.0)	1012(30)	11(1)
	H(I)-Pre	0.918(0.019)↓	1.0(0.0)	1655(18)↑	19(2)↑
	H(I)-Post	0.924(0.022)↓	1.0(0.0)	777(38)↓	19(2)↑
	H(II)-RI-D	0.901(0.032)↓	1.0(0.0)	144(52)↓	52(5)↑
	H(II)-RI-F	0.925(0.025)	1.0(0.0)	76(53)↓	52(5)↑
	H(II)-TI-D	0.907(0.03)↓	1.0(0.001)	162(61)↓	55(8)↑
	H(II)-TI-F	0.927(0.023)	1.0(0.0)	86(63)↓	55(8)↑
B	ExSTraCS	0.828(0.028)	1.0(0.0)	1333(30)	13(1)
	H(I)-Pre	0.829(0.029)↑	1.0(0.0)	1483(55)↑	22(3)↑
	H(I)-Post	0.826(0.029)↓	1.0(0.0)	737(45)↓	22(3)↑
	H(II)-RI-D	0.826(0.029)↓	0.999(0.002)↓	64(36)↓	54(7)↑
	H(II)-RI-F	0.831(0.029)↑	1.0(0.002)↓	23(22)↓	54(7)↑
	H(II)-TI-D	0.826(0.028)↓	0.998(0.004)↓	66(48)↓	58(8)↑
	H(II)-TI-F	0.83(0.028)↑	0.999(0.002)↓	25(29)↓	58(8)↑
C	ExSTraCS	0.8(0.043)	1.0(0.0)	1103(28)	12(2)
	H(I)-Pre	0.814(0.042)↑	1.0(0.0)	1580(22)↑	18(1)↑
	H(I)-Post	0.738(0.048)↓	1.0(0.0)	548(34)↓	18(1)↑
	H(II)-RI-D	0.802(0.047)↑	0.993(0.008)↓	20(7)↓	50(6)↑
	H(II)-RI-F	0.807(0.046)↑	0.994(0.007)↓	13(4)↓	50(6)↑
	H(II)-TI-D	0.804(0.043)↑	0.994(0.007)↓	21(7)↓	50(8)↑
	H(II)-TI-F	0.809(0.044)↑	0.994(0.007)↓	13(5)↓	50(8)↑
D	ExSTraCS	0.681(0.032)	1.0(0.0)	1213(26)	14(2)
	H(I)-Pre	0.673(0.044)↓	1.0(0.0)	1614(37)↑	21(3)↑
	H(I)-Post	0.67(0.049)↓	1.0(0.0)	737(73)↓	21(3)↑
	H(II)-RI-D	0.685(0.045)↑	1.0(0.002)↓	147(58)↓	51(7)↑
	H(II)-RI-F	0.699(0.043)↑	1.0(0.002)↓	87(43)↓	51(7)↑
	H(II)-TI-D	0.684(0.042)↑	0.999(0.003)↓	157(55)↓	55(7)↑
	H(II)-TI-F	0.698(0.041)↑	0.999(0.002)↓	99(51)↓	55(7)↑
E	ExSTraCS	0.617(0.038)	1.0(0.0)	1368(28)	15(1)
	H(I)-Pre	0.594(0.035)↓	1.0(0.0)	1689(38)↑	23(2)↑
	H(I)-Post	0.595(0.036)↓	1.0(0.0)	842(72)↓	23(2)↑
	H(II)-RI-D	0.605(0.042)↓	1.0(0.002)↓	85(31)↓	54(6)↑
	H(II)-RI-F	0.629(0.038)↑	1.0(0.001)↓	46(29)↓	54(6)↑
	H(II)-TI-D	0.607(0.037)↓	1.0(0.002)↓	100(35)↓	58(7)↑
	H(II)-TI-F	0.632(0.034)↑	1.0(0.001)↓	51(32)↓	58(7)↑
F	ExSTraCS	0.498(0.05)	1.0(0.0)	1338(49)	16(2)
	H(I)-Pre	0.608(0.038)↑	1.0(0.0)	1617(27)↑	23(3)↑
	H(I)-Post	0.578(0.031)↑	1.0(0.0)	927(94)↓	23(3)↑
	H(II)-RI-D	0.616(0.043)↑	0.992(0.012)↓	83(56)↓	65(13)↑
	H(II)-RI-F	0.631(0.04)↑	0.992(0.012)↓	57(37)↓	65(13)↑
	H(II)-TI-D	0.624(0.048)↑	0.991(0.013)↓	107(76)↓	68(13)↑
	H(II)-TI-F	0.638(0.047)↑	0.991(0.013)↓	74(59)↓	68(13)↑

given it's use of batch-rule-evaluation, HEROS Phase I required larger run times than ExSTraCS, and adding Phase II more than tripled overall run time, (4) HEROS Phase II *target* initialization (TI) slightly outperformed *random* initialization (RI) in some GAMETES datasets, but overall the differences were small, (5) for all datasets, selecting a 'best' model from the front (-F) yielded higher testing accuracy and lower rule count than the default method (-D), and (7) without Phase II, HEROS Phase I (i.e. H(I)-Post) yielded worse testing accuracy than ExSTraCS. This last finding is expected, given that HEROS Phase I focuses on generating a diversity of rules (rather than ideal rules for a final model).

Table 4 gives similar comparisons for MUX datasets, adding how often (out of 200 runs) the optimally compact solution was identified (see Table 2). Importantly, these results were obtained using the same hyperparameter values as for noisy GAMETES datasets (including $\nu = 1$). Key observations across clean MUX data evaluations are as follows: (1) Despite ExSTraCS having the accuracy-based rule-fitness advantage for clean problems, for MUX6, HEROS Phase II using *target* initialization (TI) consistently identified the optimally compact and accurate rule set for MUX6, while ExSTraCS achieved only near ideal average accuracy and a far larger rule count, (2) for MUX11, HEROS was still able to identify the ideal rule set 97/200 times, while ExSTraCS never does, (3) for MUX11, MUX20, MUX37, and MUX70, ExSTraCS's accuracy-fitness advantage becomes clearer (as expected) yielding higher average prediction accuracies but again with much larger rule sets. Overall HEROS performed well on these datasets despite having to explore the space of candidate rules with both high UA_r or UC_r , while ExSTraCS need only focus on high accuracy rules. Additional analysis using high accuracy pressure, i.e. $\nu = 10$ for both HEROS and ExSTraCS, dramatically improved HEROS performance on these MUX datasets, identifying ideal rule sets for all MUX6 and MUX11 runs, and in 6/200 MUX20 runs while also achieving ideal testing accuracy for all three (see Appendix A.2.1). However, ExSTraCS still achieved higher prediction accuracies for MUX37 and MUX70 albeit with much larger rule-sets, highlighting that even with $\nu = 10$, HEROS's multi-objective fitness still puts it at a disadvantage in solving these datasets efficiently. We speculate that with more unique training instances, iterations, maximum $\{P\}$ or $\{MP\}$ sizes and/or further algorithm improvements, HEROS should be capable of identifying optimal rule-sets for any of these MUX problems.

Other focused comparisons including: (1) ExSTraCS vs. HEROS (Phase I), and (2) RI vs. TI model initialization in HEROS (Phase II), are given in Appendix sections A.2.2 and A.2.3. These further (1) highlight ExSTraCS's accuracy-fitness advantage in clean problems, (2) support choosing TI over RI model initialization, and (3) identify opportunities for improvement. Appendix A.2.4 provides an illustration of how RBML models offer interpretable solutions even in noisy problems and how HEROS can yield highly compact and interpretable models for downstream application.

4 Conclusions and Future Work

This study introduced HEROS, a noise-agnostic, interpretable, rule-based supervised classification algorithm. HEROS (1) combines a variety of LCS heuristics, (2) adopts a two-phase framework separating rule from rule-set optimization, (3) introduces batch-rule

Table 4: HEROS vs. ExSTraCS on MUX Datasets with $\nu = 1$.

Data	Algorithm Config.	Testing Accuracy	Testing Coverage	Rule Count	Run Time (Minutes)	Optimal Solution
MUX6	ExSTraCS	0.999(0.005)	1.0(0.0)	415(12)	4(1)	0/200
	H(I)-Pre	0.868(0.005)↓	1.0(0.0)	332(14)↓	5(0)↑	0/200
	H(I)-Post	0.813(0.069)↓	1.0(0.0)	91(1)↓	5(0)↑	0/200
	H(II)-RI-D	0.998(0.016)	1.0(0.0)	8(1)↓	8(0)↑	180/200↑
	H(II)-RI-F	0.999(0.005)	1.0(0.0)	8(0)↓	8(0)↑	190/200↑
	H(II)-TI-D	1.0(0.0)	1.0(0.0)	8(0)↓	7(1)↑	200/200↑
MUX11	ExSTraCS	1.0(0.001)	1.0(0.0)	719(25)	10(1)	0/200
	H(I)-Pre	0.838(0.023)↓	1.0(0.0)	1243(38)↑	30(1)↑	0/200
	H(I)-Post	0.803(0.021)↓	1.0(0.0)	601(14)↓	30(1)↑	0/200
	H(II)-RI-D	0.932(0.031)↓	0.999(0.003)↓	40(13)↓	103(4)↑	0/200
	H(II)-RI-F	0.946(0.026)↓	0.999(0.003)↓	32(11)↓	103(4)↑	0/200
	H(II)-TI-D	0.993(0.013)↓	0.997(0.007)↓	19(4)↓	108(14)↑	89/200↑
MUX20	ExSTraCS	0.986(0.006)	1.0(0.0)	837(21)	14(2)	0/200
	H(I)-Pre	0.694(0.038)↓	1.0(0.0)	1469(91)↑	52(8)↑	0/200
	H(I)-Post	0.678(0.034)↓	1.0(0.0)	1019(56)↑	52(8)↑	0/200
	H(II)-RI-D	0.782(0.044)↓	0.999(0.002)↓	130(27)↓	246(36)↑	0/200
	H(II)-RI-F	0.798(0.038)↓	0.999(0.002)↓	101(27)↓	246(36)↑	0/200
	H(II)-TI-D	0.816(0.06)↓	0.995(0.009)↓	112(29)↓	253(14)↑	0/200
MUX37	ExSTraCS	0.901(0.022)	1.0(0.0)	1017(28)	17(0)	0/200
	H(I)-Pre	0.539(0.017)↓	1.0(0.0)	1272(101)↑	57(2)↑	0/200
	H(I)-Post	0.537(0.016)↓	1.0(0.0)	866(82)↓	57(2)↑	0/200
	H(II)-RI-D	0.556(0.02)↓	1.0(0.0)	200(37)↓	284(21)↑	0/200
	H(II)-RI-F	0.575(0.016)↓	1.0(0.0)	101(42)↓	284(21)↑	0/200
	H(II)-TI-D	0.555(0.019)↓	1.0(0.0)	202(40)↓	275(35)↑	0/200
MUX70	ExSTraCS	0.529(0.011)	1.0(0.0)	1104(46)	17(2)	0/200
	H(I)-Pre	0.518(0.01)↓	1.0(0.0)	1502(39)↑	104(14)↑	0/200
	H(I)-Post	0.516(0.009)↓	1.0(0.0)	956(39)↓	104(14)↑	0/200
	H(II)-RI-D	0.521(0.011)↓	1.0(0.0)	316(58)↓	670(101)↑	0/200
	H(II)-RI-F	0.534(0.009)↑	1.0(0.0)	139(69)↓	670(101)↑	0/200
	H(II)-TI-D	0.522(0.01)↓	1.0(0.0)	313(59)↓	687(90)↑	0/200
MUX70	H(II)-TI-F	0.534(0.009)↑	1.0(0.0)	140(72)↓	687(90)↑	0/200

evaluation adaptations, and (4) proposes the use of two distinct, adaptive, Pareto-front-based multi-objective optimization strategies to tackle diverse and challenging problems. Proof-of-principle results found that HEROS outperformed ExSTraCS in compactly solving a variety of noisy simulated problems and could find ideally compact solutions for (at-least) lower-order clean MUX problems, without adjusting hyperparameters. We also demonstrate the value in considering alternative model initializations strategies, and examining all Pareto-optimal models as candidate solutions to obtain more compact and less over-fit models in noisy problems. In prioritizing model interpretability and adaptability, the current limitations of HEROS include: (1) higher computational expense, (2) Phase II reliance on rules discovered during Phase I, and (3) less efficient learning than algorithms relying on correct assumptions.

Next steps will focus on broadening the spectrum of datasets, data scale, feature-types, algorithms (e.g. BioHEL [1, 9]), and computational resources utilized to evaluate and compare HEROS. Ongoing algorithm development will target phase alternation [13, 16] alternative model initialization strategies, efficiency and parallelization strategies, and extension to multi-class, quantitative, and survival outcomes (e.g. [42]). We believe this work demonstrates significant promise for a broad range of real-world applications that require truly interpretable model decision-making.

Acknowledgments

The study was supported by Cedars Sinai Medical Center and NIH grants R01 A1173095, U01 AG066833 and P30 AG0373105. Special thanks to Robert Zhang, Dr. John Holmes and Dr. Jason Moore.

References

- [1] Jaume Bacardit, Edmund K Burke, and Natalio Krasnogor. 2009. Improving the scalability of rule-based evolutionary learning. *Memetic Computing* 1, 1 (2009), 55–67.
- [2] Jaume Bacardit and Natalio Krasnogor. 2009. A mixed discrete-continuous attribute list representation for large scale classification domains. In *Proceedings of the 11th Annual conference on Genetic and evolutionary computation*. 1155–1162.
- [3] Ester Bernadó-Mansilla and Josep M Garrell-Guiu. 2003. Accuracy-based learning classifier systems: models, analysis and applications to classification tasks. *Evolutionary computation* 11, 3 (2003), 209–238.
- [4] William W Cohen. 1995. Fast effective rule induction. In *Machine learning proceedings 1995*. Elsevier, 115–123.
- [5] Leandro Nunes De Castro and Jonathan Timmis. 2002. *Artificial immune systems: a new computational intelligence approach*. Springer Science & Business Media.
- [6] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE transactions on evolutionary computation* 6, 2 (2002), 182–197.
- [7] Phillip William Dixon, Dawid Wolfe Corne, and Martin John Oates. 2002. A ruleset reduction algorithm for the XCS learning classifier system. In *International workshop on learning classifier systems*. Springer, 20–29.
- [8] Dheeru Dua and Casey Graff. 2017. UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml>
- [9] Maria A Franco, Natalio Krasnogor, and Jaume Bacardit. 2010. Speeding up the evaluation of evolutionary learning systems using GPGPUs. In *Proceedings of the 12th annual conference on Genetic and evolutionary computation*. 920–1046.
- [10] Chunsheng Fu and Lawrence Davis. 2002. A modified classifier system compaction algorithm. In *Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation*. 920–925.
- [11] Nyoman Gunantara. 2018. A review of multi-objective optimization: Methods and its applications. *Cogent Engineering* 5, 1 (2018), 1502242.
- [12] Michael Heider, Maximilian Krischan, Roman Sraj, and Jörg Hähner. 2024. Exploring Self-Adaptive Genetic Algorithms to Combine Compact Sets of Rules. In *2024 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 1–8.
- [13] Michael Heider, Helena Stegherr, Roman Sraj, David Pätzl, Jonathan Wurth, and Jörg Hähner. 2023. SupRB in the context of rule-based machine learning methods: A comparative study. *Applied Soft Computing* 147 (2023), 110706.
- [14] Tin Kam Ho. 1995. Random decision forests. In *Proceedings of 3rd international conference on document analysis and recognition*, Vol. 1. IEEE, 278–282.
- [15] Muhammad Iqbal, Will N Browne, and Mengjie Zhang. 2013. Reusing building blocks of extracted knowledge to solve complex, large-scale boolean problems. *IEEE Transactions on Evolutionary Computation* 18, 4 (2013), 465–480.
- [16] Hisao Ishibuchi, Tomoharu Nakashima, and Tetsuya Kuroda. 2000. A hybrid fuzzy GBML algorithm for designing compact fuzzy rule-based classification systems. In *Ninth IEEE International Conference on Fuzzy Systems. FUZZ-IEEE 2000 (Cat. No. 00CH37063)*, Vol. 2. IEEE, 706–711.
- [17] Tim Kovacs. 1999. Deletion schemes for classifier systems. In *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation-Volume 1*. 329–336.
- [18] Tim Kovacs. 2001. Two views of classifier systems. In *International Workshop on Learning Classifier Systems*. Springer, 74–87.
- [19] Zachary C Lipton. 2018. The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue* 16, 3 (2018), 31–57.
- [20] Yi Liu, Will N Browne, and Bing Xue. 2019. Absumption to complement subsumption in learning classifier systems. In *Proceedings of the genetic and evolutionary computation conference*. 410–418.
- [21] Yi Liu, Will N Browne, and Bing Xue. 2021. A comparison of learning classifier systems' rule compaction algorithms for knowledge visualization. *ACM Transactions on Evolutionary Learning and Optimization* 1, 3 (2021), 1–38.
- [22] Trung B Nguyen, Will N Browne, and Mengjie Zhang. 2022. ConCS: A Continual Classifier System for Continual Learning of Multiple Boolean Problems. *IEEE Transactions on Evolutionary Computation* 27, 4 (2022), 1057–1071.
- [23] Jaume Bacardit Peñarroya. 2004. *Pittsburgh genetic-based machine learning in the data mining era: representations, generalization, and run-time*. Ph.D. Dissertation. Universitat Ramon Llull.
- [24] Cynthia Rudin. 2019. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence* 1, 5 (2019), 206–215.
- [25] Roman Sergienko and Eugene Semenkin. 2013. Michigan and Pittsburgh methods combination for fuzzy classifier design with coevolutionary algorithm. In *2013 IEEE Congress on Evolutionary Computation*. IEEE, 3252–3259.
- [26] Jie Tan, Jason Moore, and Ryan Urbanowicz. 2013. Rapid rule compaction strategies for global knowledge discovery in a supervised learning classifier system. In *Artificial Life Conference Proceedings*. Citeseer, 110–117.
- [27] Ryan Urbanowicz, Ambrose Granizo-Mackenzie, and Jason Moore. 2012. Instance-linked attribute tracking and feedback for michigan-style supervised learning classifier systems. In *Proceedings of the 14th annual conference on Genetic and evolutionary computation*. 927–934.
- [28] Ryan Urbanowicz and Jason Moore. 2015. Retooling fitness for noisy problems in a supervised Michigan-style learning classifier system. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*. 591–598.
- [29] Ryan J Urbanowicz, Gediminas Bertasius, and Jason H Moore. 2014. An extended michigan-style learning classifier system for flexible supervised learning, classification, and data mining. In *Parallel Problem Solving from Nature-PPSN XIII: 13th International Conference, Ljubljana, Slovenia, September 13-17, 2014. Proceedings 13*. Springer, 211–221.
- [30] Ryan J. Urbanowicz and Will N. Browne. 2017. *Introduction to Learning Classifier Systems*. Springer Berlin Heidelberg, Berlin, Heidelberg. doi:10.1007/978-3-662-55007-6
- [31] Ryan J Urbanowicz, Delaney Granizo-Mackenzie, and Jason H Moore. 2012. Using expert knowledge to guide covering and mutation in a michigan style learning classifier system to detect epistasis and heterogeneity. In *Parallel Problem Solving from Nature-PPSN XII: 12th International Conference, Taormina, Italy, September 1-5, 2012, Proceedings, Part I 12*. Springer, 266–275.
- [32] Ryan J Urbanowicz, Jeff Kiralis, Nicholas A Sinnott-Armstrong, Tamra Heberling, Jonathan M Fisher, and Jason H Moore. 2012. GAMETES: a fast, direct algorithm for generating pure, strict, epistatic models with random architectures. *BioData Mining* 5, 1 (2012), 1–14.
- [33] Ryan J Urbanowicz, Christopher Lo, John H Holmes, and Jason H Moore. 2018. Attribute tracking: strategies towards improved detection and characterization of complex associations. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 553–560.
- [34] Ryan J Urbanowicz and Jason H Moore. 2009. Learning classifier systems: a complete introduction, review, and roadmap. *Journal of Artificial Evolution and Applications* 2009 (2009).
- [35] Ryan J Urbanowicz and Jason H Moore. 2015. ExSTraCS 2.0: description and evaluation of a scalable learning classifier system. *Evolutionary Intelligence* 8, 2 (2015), 89–116.
- [36] Ryan J Urbanowicz, Randal S Olson, and Jason H Moore. 2016. Pareto inspired multi-objective rule fitness for noise-adaptive rule-based machine learning. In *International Conference on Parallel Problem Solving from Nature*. Springer, 514–524.
- [37] Ryan J Urbanowicz, Randal S Olson, Peter Schmitt, Melissa Meeker, and Jason H Moore. 2018. Benchmarking relief-based feature selection methods for bioinformatics data mining. *Journal of biomedical informatics* 85 (2018), 168–188.
- [38] Jonathan Waring, Charlotta Lindvall, and Renato Umeton. 2020. Automated machine learning: Review of the state-of-the-art and opportunities for healthcare. *Artificial intelligence in medicine* 104 (2020), 101822.
- [39] Donald Arthur Waterman and Frederick Hayes-Roth. 1978. *Pattern-directed inference systems*. Academic press.
- [40] Stewart W Wilson. 1995. Classifier fitness based on accuracy. *Evolutionary computation* 3, 2 (1995), 149–175.
- [41] Stewart W Wilson. 2001. Compact rulesets from XCSL. In *International Workshop on Learning Classifier Systems*. Springer, 197–208.
- [42] Alexa Woodward, Harsh Bandhey, Jason H Moore, and Ryan J Urbanowicz. 2024. Survival-LCS: A Rule-Based Machine Learning Approach to Survival Analysis. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 431–439.
- [43] Chengqi Zhang and Shichao Zhang. 2002. *Association rule mining: models and algorithms*. Springer.
- [44] Robert Zhang, Rachael Stolzenberg-Solomon, Shannon M Lynch, and Ryan J Urbanowicz. 2021. LCS-DIVE: An Automated Rule-based Machine Learning Visualization Pipeline for Characterizing Complex Associations in Classification. *arXiv preprint arXiv:2104.12844* (2021).
- [45] Robert Zhang and Ryan Urbanowicz. 2022. scikit-ExSTraCS. <https://github.com/UrbsLab/scikit-ExSTraCS>.

Received 29 January 2025; accepted 19 March 2025